

# ALGORITHMS THAT CHANGED THE WORLD

## PAGERANK - FINDING A NEEDLE IN THE OCEAN

---

Dr Chunbo Luo

Updated: 2026/09/24  
Lecture\_PageRank

*University of Exeter*



University  
of Exeter

## Contents

- **Introduction**
- Search engine and challenges
- Google PageRank algorithm
  - The hyperlink trick
  - Random surfer model
- Practical issues
- Summary

## Three well-known garage stories

- In 1939, Hewlett-Packard was established in the garage at 367 Addison Avenue in Palo Alto, California.
- In 1976, Steve Jobs and Steve Wozniak began assembling early **APPLE** computers at Jobs's family home in Los Altos, California.
- In 1998, Larry Page and Sergey Brin operated **GOOGLE** from a garage in Menlo Park, California.

These sites became symbols of Silicon Valley entrepreneurship.

Why do garages have magic power?

## Contents

- Introduction
- **Search engine and challenges**
- Google PageRank algorithm
  - The hyperlink trick
  - Random surfer model
- Practical issues
- Summary

Challenges of building a web search engine:

- An enormous number of web pages must be discovered and indexed.
- Web pages are not ordered or indexed by a single central authority.
- There is no universal classification scheme for web pages.
- The Web contains many duplicate and near-duplicate pages.

# Success of Google: PageRank

One of Google's foundational algorithms is *PageRank*.

Two principles of the PageRank algorithm:

- Hyperlink analysis
- Random surfer model



(a) Larry Page



(b) Sergey Brin

# PageRank algorithm

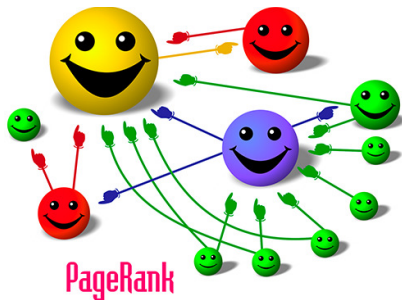


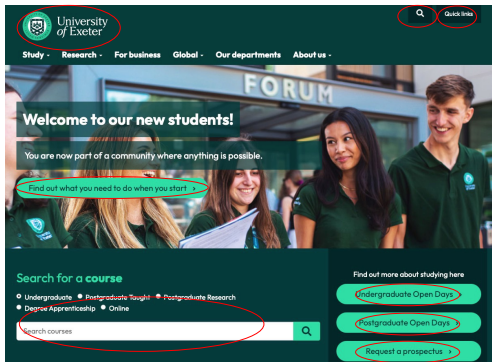
Figure: How does Google work?

## Contents

- Introduction
- Search engine and challenges
- **Google PageRank algorithm**
  - The hyperlink trick
  - Random surfer model
- Practical issues
- Summary

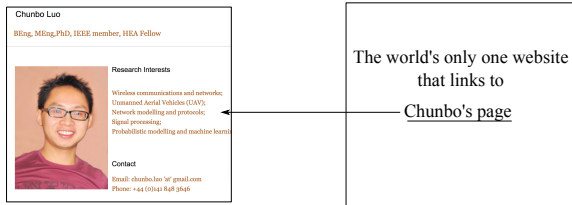
# PageRank algorithm

A web page may have many *hyperlinks*.



# PageRank algorithm

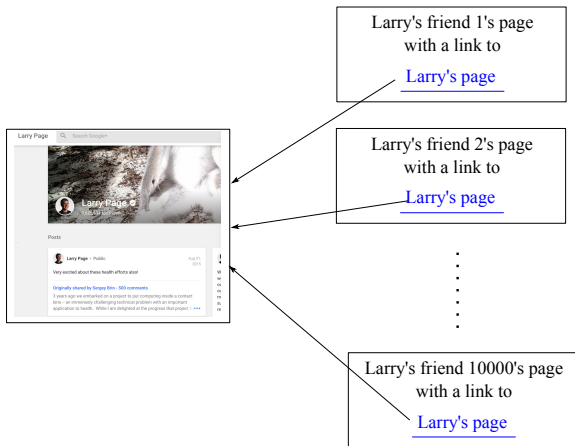
## Hyperlinks: number of incoming links



The number of incoming links provides a first approximation of a page's popularity.

# PageRank algorithm

## Hyperlink analysis: number of incoming links



## **Hyperlink analysis: number of incoming links**

- As a first approximation, pages can be ranked by their **number of incoming links**.

## Hyperlink analysis: link quality

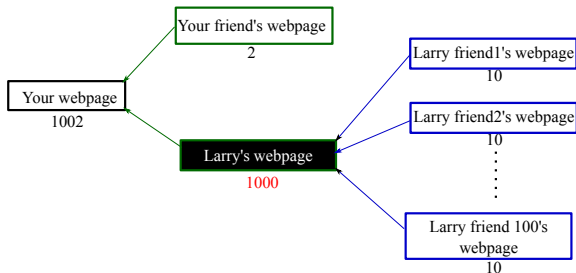
**Example:** Suppose you write a web page about the *PageRank* algorithm and two pages link to it: mine and Larry Page's.

### Questions:

- Which link should be given more **weight**?
- How can a search engine determine the weight of each link?

# PageRank algorithm

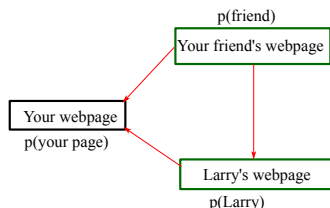
## Hyperlink analysis: link quality



# PageRank algorithm

**Let us model link-based importance.**

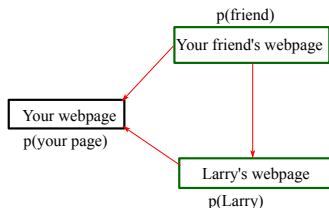
Suppose the entire Web contains only the following three pages:



$p(\text{your page})$ ,  $p(\text{friend})$  and  $p(\text{Larry})$  are the ranks of your page, your friend's page and Larry's page.

# PageRank algorithm

Let us model link-based importance.



We obtain your page's rank from the two pages that link to it:

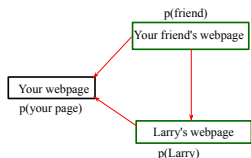
$$p(\text{your page}) = \beta \times p(\text{friend}) + \gamma \times p(\text{Larry}).$$

Here,  $\beta$  and  $\gamma$  are weighting factors.

# PageRank algorithm

Let us model link-based importance.

If a page has  $N$  outgoing links, its rank is split equally among them. Hence  $\beta = \frac{1}{2}$  and  $\gamma = 1$  in this example.



Similarly, for the other two pages,

$$p(\text{friend}) = 0,$$

$$p(\text{Larry}) = \frac{1}{2}p(\text{friend}).$$

# PageRank algorithm

In a matrix form,

$$\underbrace{\begin{bmatrix} p(\text{your page}) \\ p(\text{friend}) \\ p(\text{Larry}) \end{bmatrix}}_{P_1} = \underbrace{\begin{bmatrix} 0 & \frac{1}{2} & 1 \\ 0 & 0 & 0 \\ 0 & \frac{1}{2} & 0 \end{bmatrix}}_H \underbrace{\begin{bmatrix} p(\text{your page}) \\ p(\text{friend}) \\ p(\text{Larry}) \end{bmatrix}}_{P_0}$$

$$\boxed{P_1 = HP_0}$$

# PageRank algorithm

A simple choice is the uniform initial vector:

$$p(\text{your page}) = p(\text{friend}) = p(\text{Larry}) = \frac{1}{3}.$$

If there are  $N$  pages in the network, the initial ranks are all  $\frac{1}{N}$ .

# PageRank algorithm

Summary (the basic version):

- Step 1: Initialise  $P_0$  and construct  $H$ .
- Step 2: Iterate:  $P_{m+1} = HP_m$ .
- Step 3: Stop when  $\|P_{m+1} - P_m\|$  is below a tolerance.

*Question: Termination conditions?*

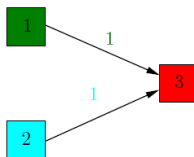
This basic iteration may lose rank mass or fail to converge until dangling pages and closed link traps are handled.

## Contents

- Introduction
- Search engine and challenges
- Google PageRank algorithm
  - The hyperlink trick
  - Random surfer model
- **Practical issues**
- Summary

# Practical challenges

1. Dangling pages have no outgoing links.



2. Closed link traps retain rank and make oscillation.

# The dangling-page problem

$$H = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 1 & 1 & 0 \end{bmatrix}, \quad P_0 = \begin{bmatrix} 1/3 \\ 1/3 \\ 1/3 \end{bmatrix}.$$

$$P_1 = HP_0 = \begin{bmatrix} 0 \\ 0 \\ 2/3 \end{bmatrix},$$

$$P_2 = HP_1 = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}.$$

The total rank falls from 1 to 2/3 and then to 0 because page 3 has no outgoing link. This loss of probability mass is the dangling-page problem.

# The dangling-page problem

**Solution:** Treat a dangling page as linking uniformly to all  $N$  pages. Let  $a_j = 1$  when column  $j$  of  $H$  is all zero, and  $a_j = 0$  otherwise. The repaired transition matrix is

$$A = H + \frac{1}{N} \mathbf{1} a^T.$$

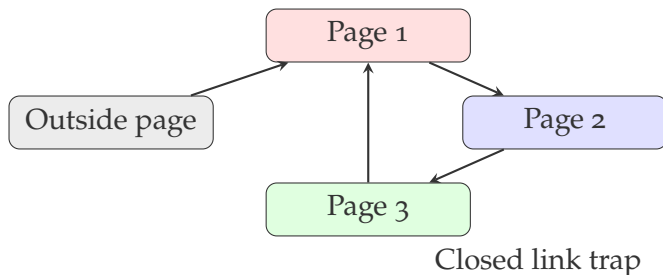
**Example:** The third column is dangling, so replace it with  $1/3$  in every row:

$$\begin{bmatrix} p_1(1) \\ p_1(2) \\ p_1(3) \end{bmatrix} = \begin{bmatrix} 0 & 0 & \textcolor{red}{1/3} \\ 0 & 0 & \textcolor{red}{1/3} \\ 1 & 1 & \textcolor{red}{1/3} \end{bmatrix} \begin{bmatrix} 1/3 \\ 1/3 \\ 1/3 \end{bmatrix} = \begin{bmatrix} 1/9 \\ 1/9 \\ 7/9 \end{bmatrix}$$

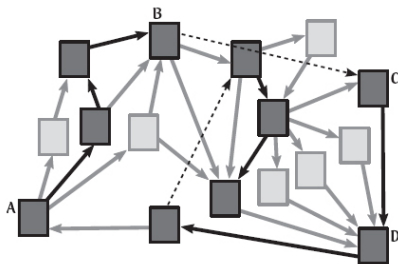
# The closed-cycle problem

## What is the closed-cycle problem?

A closed group of pages can trap rank: once enters the group, no link lets it leave.



# Solution: Random surfer model



A random surfer either follows an outgoing link or, with a small probability, teleports to a page chosen uniformly at random.

*Why does the Random Surfer Model work?*

*Illustration: John MacCormick.*

# Random surfer model

Let  $d$  denote the damping factor: the probability of following a link (often 0.85). The probability of teleporting is therefore  $1 - d$  (often 0.15):

$$B = \frac{1}{N} \mathbf{1}\mathbf{1}^T, \quad G = dA + (1 - d)B.$$

$B$  assigns equal teleportation probability to every page, and  $G$  is the Google matrix.

For  $0 < d < 1$ ,  $G$  is positive and column-stochastic. It therefore has a unique stationary probability vector, and the power iteration converges to it.

Question: why is every entry of  $B$  equal to  $\frac{1}{N}$ ?

## Complete algorithm steps

- Step 0: Generate the hyperlink matrix  $H$ .
- Step 1: Repair every dangling column. Let  $a_j = 1$  when column  $j$  of  $H$  is all zero, and  $a_j = 0$  otherwise:

$$A = H + \frac{1}{N} \mathbf{1} a^\top.$$

- Step 2: Add teleportation, where  $B = \frac{1}{N} \mathbf{1} \mathbf{1}^\top$ :

$$G = dA + (1 - d)B.$$

- Step 3: Apply the power iteration until convergence:

$$P_{k+1} = GP_k, \quad \|P_{k+1} - P_k\|_1 \leq \varepsilon.$$

# Algorithm Description

---

## Algorithm PageRank

---

- 1: **Input:** Transition Matrix  $H$ , damping factor  $d$  and stopping error  $e$ .
  - 2: **Output:** Ranks of all pages:  $P$ .
  - 3: Generate the initial rank vector  $P$ .
  - 4: Generate a vector  $\tilde{P}$ , which has arbitrary large entries.
  - 5: Calculate the Google Matrix  $G = d * A + (1 - d) * B$ .
  - 6: **while**  $\|\tilde{P} - P\| > e$  **do**
    - 7:      $\tilde{P} = P$ ;
    - 8:      $P = G * \tilde{P}$ ;
  - 9: **end while**
-

---

## Algorithm PageRank

---

- 1: ...
  - 2: Generate the initial rank vector  $P$ .
  - 3: Generate a vector  $\tilde{P}$ , which has arbitrary large entries.
  - 4: Calculate the Google Matrix  $G = d * A + (1 - d) * B$ .
  - 5: **while**  $\|\tilde{P} - P\| > \epsilon$  **do** ▷ Iteration:  $M$  times
  - 6:      $\tilde{P} = P$ ; ▷ 1
  - 7:      $P = G * P$ ; ▷  $N^2$  multiplications
  - 8: **end while**
- 

Time complexity:  $O(MN^2)$ .

# C++ Implementation

---

```
1    // Calculate G
2    for(int i=0; i< SIZE; i++){
3        for(int j=0; j< SIZE; j++){
4            G[i][j]= d*A[i][j] + (1-d) * B[i][j];
5        }
6    }
7    // Page rank update
8    double error = e + 1.0;
9    int it = 0;
10   while (error > e && it < iteration){
11       NP = P;
12       MM(G, NP, P); // Separate input/output: P = G * NP
13       error = 0.0;
14       for(int i=0; i< SIZE; i++){
15           error += std::abs(P[i] - NP[i]);
16       }
17       it++;
18   }
```

---

## Contents

- Introduction
- Search engine and challenges
- Google PageRank algorithm
  - The hyperlink trick
  - Random surfer model
- Practical issues
- **Summary**

# PageRank Algorithm

**Note:** The famous paper by Brin and Page<sup>1</sup> introduced a way to calculate the rank of a single page below:

The rank of Page A is:

$$P_{n+1}(A) = \frac{(1-d)}{N} + d \left( \frac{P_n(1)}{C(1)} + \dots + \frac{P_n(N)}{C(N)} \right)$$

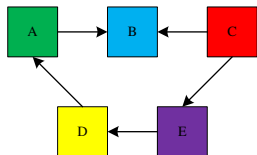
where  $C(i)$  is the number of outbound links of a page  $i$ . (If  $i$  is a dangling page, it will be assumed to link to all the pages, thus  $C(i) = N$ .)

**QUESTION:** Work out the algorithm expressed in the form of Matrices. Will the two have the same results?

---

<sup>1</sup>The Anatomy of a Large-Scale Hypertextual Web Search Engine

# Another example



1. Construct the hyperlink matrix.

Figure: Damping factor

$$d = \frac{4}{5}$$

$$H = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 \\ 1 & 0 & \frac{1}{2} & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & \frac{1}{2} & 0 & 0 \end{bmatrix}$$

## Another example (continued)

2. Repair the dangling page B by replacing the second column of  $H$  with  $1/5$ .

$$A = \begin{bmatrix} 0 & \frac{1}{5} & 0 & 1 & 0 \\ 1 & \frac{1}{5} & \frac{1}{2} & 0 & 0 \\ 0 & \frac{1}{5} & 0 & 0 & 0 \\ 0 & \frac{1}{5} & 0 & 0 & 1 \\ 0 & \frac{1}{5} & \frac{1}{2} & 0 & 0 \end{bmatrix}$$

## Another example (continued)

3. Generate matrix  $B$  with every entry equal to  $1/N$ , where  $N = 5$ :

$$B = \begin{bmatrix} \frac{1}{5} & \frac{1}{5} & \frac{1}{5} & \frac{1}{5} & \frac{1}{5} \\ \frac{1}{5} & \frac{1}{5} & \frac{1}{5} & \frac{1}{5} & \frac{1}{5} \\ \frac{1}{5} & \frac{1}{5} & \frac{1}{5} & \frac{1}{5} & \frac{1}{5} \\ \frac{1}{5} & \frac{1}{5} & \frac{1}{5} & \frac{1}{5} & \frac{1}{5} \\ \frac{1}{5} & \frac{1}{5} & \frac{1}{5} & \frac{1}{5} & \frac{1}{5} \end{bmatrix}$$

## Another example (continued)

4. Calculate the Google matrix:

$$G = d * A + (1 - d) * B = \frac{4}{5}A + \frac{1}{5}B =$$

$$\begin{bmatrix} 0 & \frac{4}{25} & 0 & \frac{4}{5} & 0 \\ \frac{4}{5} & \frac{4}{25} & \frac{2}{5} & 0 & 0 \\ 0 & \frac{4}{25} & 0 & 0 & 0 \\ 0 & \frac{4}{25} & 0 & 0 & \frac{4}{5} \\ 0 & \frac{4}{25} & \frac{2}{5} & 0 & 0 \end{bmatrix} + \begin{bmatrix} \frac{1}{25} & \frac{1}{25} & \frac{1}{25} & \frac{1}{25} & \frac{1}{25} \\ \frac{1}{25} & \frac{1}{25} & \frac{1}{25} & \frac{1}{25} & \frac{1}{25} \\ \frac{1}{25} & \frac{1}{25} & \frac{1}{25} & \frac{1}{25} & \frac{1}{25} \\ \frac{1}{25} & \frac{1}{25} & \frac{1}{25} & \frac{1}{25} & \frac{1}{25} \\ \frac{1}{25} & \frac{1}{25} & \frac{1}{25} & \frac{1}{25} & \frac{1}{25} \end{bmatrix} = \begin{bmatrix} \frac{1}{25} & \frac{1}{5} & \frac{1}{25} & \frac{21}{25} & \frac{1}{25} \\ \frac{21}{25} & \frac{1}{5} & \frac{11}{25} & \frac{1}{25} & \frac{1}{25} \\ \frac{1}{25} & \frac{1}{5} & \frac{1}{25} & \frac{1}{25} & \frac{1}{25} \\ \frac{1}{25} & \frac{1}{5} & \frac{1}{25} & \frac{1}{25} & \frac{21}{25} \\ \frac{1}{25} & \frac{1}{5} & \frac{11}{25} & \frac{1}{25} & \frac{1}{25} \end{bmatrix}$$

## Knowledge extension

Citation networks provide another directed graph. Citation count is an in-degree measure; a PageRank-like method also weights a citation by the score and out-degree of the citing paper.

### HYPOTHESIS

Suppose my book on “Programming” has been cited twice, whereas Donald Knuth’s *The Art of Computer Programming* has 10,000 citations. A citation-based method should then rank Knuth’s work more highly.

**PROBLEM: CAN YOU DERIVE AN APPROACH FOR RANKING ACADEMIC PAPERS?**

- S. Brin and L. Page, “The Anatomy of a Large-Scale Hypertextual Web Search Engine”, *Computer Networks and ISDN Systems*, 30(1–7), 107–117, 1998.  
doi:10.1016/S0169-7552(98)00110-X.
- L. Page, S. Brin, R. Motwani and T. Winograd, “The PageRank Citation Ranking: Bringing Order to the Web”, Stanford InfoLab Technical Report, Technical Report 1999-66, 1999. Online copy.

# Further questions

- What is the PageRank algorithm?
- Next lecture: Which smart algorithms help you visit websites all over the world?
- Questions?